NTP 2026 - SpaceSec Shield

Phase 3 Final Technical Report

Security Layer for CubeSat Communications

Prepared by: Diala Sami

Program: Cyber Security - Middle East University (MEU), Jordan

Submission Context: Phase 3 of 5 - Security Layer Implementation

Date: May 2026

Core implementation: X25519 ECDH + HKDF-SHA256 + ChaCha20-Poly1305 AEAD + timestamp/nonce anti-replay over UDP.

Executive Summary

This report documents the implementation and validation of Phase 3 of SpaceSec Shield: a lightweight security layer for CubeSat-style ground-to-satellite communication. The implemented prototype secures command packets exchanged between a Ground Station and a Satellite Node over UDP. It uses an ephemeral X25519 Diffie-Hellman handshake to establish a per-session shared secret, HKDF-SHA256 to derive a 256-bit session key, and ChaCha20-Poly1305 authenticated encryption to protect the command payload and authenticate the packet header.

The security layer was tested against normal secure exchange, ciphertext tampering, authentication tag tampering, header/AAD tampering, replay using duplicate nonces, expired timestamps, and live UDP packet capture through Wireshark. The test evidence shows that valid encrypted commands are accepted, replayed packets are rejected, tampered packets are discarded before command execution, and plaintext commands do not appear in captured packet bytes.

The result is a working Phase 3 security layer that satisfies the project criteria for confidentiality, integrity, authenticity, and replay resistance while preserving a lightweight design suitable for resource-constrained CubeSat environments.

Report Structure

1. Introduction

CubeSats are small, modular satellites that reduce cost and development complexity compared with traditional large satellites. A 1U CubeSat is commonly defined as a 10 cm cube, and larger systems such as 2U, 3U, 6U, and 12U are formed by combining standardized units (Cal Poly CubeSat Program, 2022). Their low cost and modularity make them attractive for universities, research teams, startups, and distributed space missions.

However, CubeSats operate under strict size, weight, and power constraints. These SWaP constraints limit processor performance, memory capacity, battery availability, and radio bandwidth. Therefore, security mechanisms designed for traditional terrestrial systems may be too heavy for CubeSat nodes. SpaceSec Shield addresses this gap by implementing a lightweight security layer that protects satellite command traffic without requiring additional hardware.

Phase 3 focuses specifically on cybersecurity protection for the command channel. The objective is not only to encrypt data, but also to authenticate packets, prevent command injection, reject replayed packets, and preserve the freshness of commands. The implementation was validated in a Kali Linux environment using C++, OpenSSL, UDP sockets, and Wireshark packet analysis.

2. Problem Statement

A CubeSat command link can be targeted by attackers who capture, modify, replay, or forge packets. If commands are transmitted without adequate protection, an attacker may observe operational data, inject unauthorized commands, resend old commands, or impersonate a legitimate ground station. These risks are severe because even a single accepted malicious command may affect satellite attitude, payload operation, imaging tasks, or mission availability.

The main problem solved by this phase is: how can the Ground Station and Satellite Node communicate securely using lightweight cryptographic protection while maintaining compatibility with constrained CubeSat resources?

- Unprotected command payloads can be read by an eavesdropper.
- Unauthenticated packets can enable command injection or spoofing.
- Captured valid packets can be replayed to repeat old satellite actions.
- Traditional heavyweight security stacks may exceed the processing or energy limits of embedded satellite nodes.

- A standalone hash is insufficient for authentication because it does not prove that the sender has the session key.

3. Objectives

The implemented Phase 3 objectives are listed below.

- Establish a unique per-session symmetric key without transmitting the key over the network.
- Encrypt command payloads using an AEAD algorithm suitable for low-power processors.
- Authenticate packet headers using Additional Authenticated Data (AAD).
- Reject tampered ciphertext, modified authentication tags, and altered headers.
- Reject replayed packets using a timestamp freshness window and a per-session nonce cache.
- Demonstrate secure UDP communication between Ground Station and Satellite Node.
- Provide evidence through terminal tests and Wireshark packet analysis.

4. Related Work

Authenticated encryption is a practical requirement for secure command channels because encryption alone only hides content; it does not prove that a packet was not modified. ChaCha20-Poly1305 is defined as an AEAD construction in RFC 8439, combining the ChaCha20 stream cipher with a Poly1305 authentication tag (Nir & Langley, 2018). OpenSSL provides EVP support for ChaCha20-Poly1305 using a 256-bit key, 96-bit IV/nonce, AAD support, and a 128-bit authentication tag (OpenSSL Project, n.d.).

For session establishment, X25519 is a modern elliptic-curve Diffie-Hellman function specified in RFC 7748. It enables two peers to derive the same shared secret using only exchanged public keys while keeping private keys local (Langley et al., 2016). HKDF-SHA256, specified in RFC 5869, is used to transform the raw ECDH shared secret into a clean 256-bit session key suitable for symmetric encryption (Krawczyk & Eronen, 2010).

AES-128-GCM remains a strong alternative AEAD construction and is standardized by NIST as Galois/Counter Mode (Dworkin, 2007). In this project, ChaCha20-Poly1305 was selected as the primary algorithm because it performs efficiently on processors that do not include AES acceleration. This is useful for CubeSat-like systems where the target processor may be an ARM Cortex-M class microcontroller rather than a high-performance CPU.

5. Methodology

The implementation followed a modular, test-driven approach. Each security component was implemented and validated independently, then integrated into a complete end-to-end flow. This reduced debugging complexity and produced clear evidence for each security requirement.

Module	Purpose	Implemented Evidence
handshake.cpp / handshake.hpp	Generate X25519 ephemeral keys, exchange raw public keys, derive ECDH shared secret, and apply HKDF-SHA256.	Secure handshake test with matching 32-byte session keys.
crypto.cpp / crypto.hpp	Encrypt and authenticate payloads with ChaCha20-Poly1305; verify tags before releasing plaintext.	Crypto verification test rejecting tampered ciphertext, tag, and AAD.
packet.cpp / packet.hpp	Serialize Session ID, timestamp, nonce, ciphertext, and tag into a protected packet format.	Packet structure test validating header, payload, and tag tamper rejection.
anti_replay.cpp / anti_replay.hpp	Maintain timestamp window and seen-nonce cache per session.	Anti-replay test rejecting duplicate nonce and expired timestamp.
secure_ground_station.cpp	Create session key and send encrypted command packets over UDP.	UDP demo showing valid, replay, tampered, and expired packets.
secure_satellite_node.cpp	Receive UDP packets, verify freshness, decrypt authenticated packets, and reject invalid traffic.	Terminal evidence showing accept/reject results.

The testing sequence was: standalone cryptographic verification, packet format validation, anti-replay validation, secure handshake validation, integrated end-to-end flow, live UDP communication, and Wireshark packet inspection.

6. System and Security Design

The system consists of two logical endpoints: a Ground Station and a Satellite Node. The Ground Station initiates a secure session, sends encrypted command packets, and simulates normal and attack traffic. The Satellite Node receives packets, verifies security properties, and executes commands only after successful authentication and replay checks.

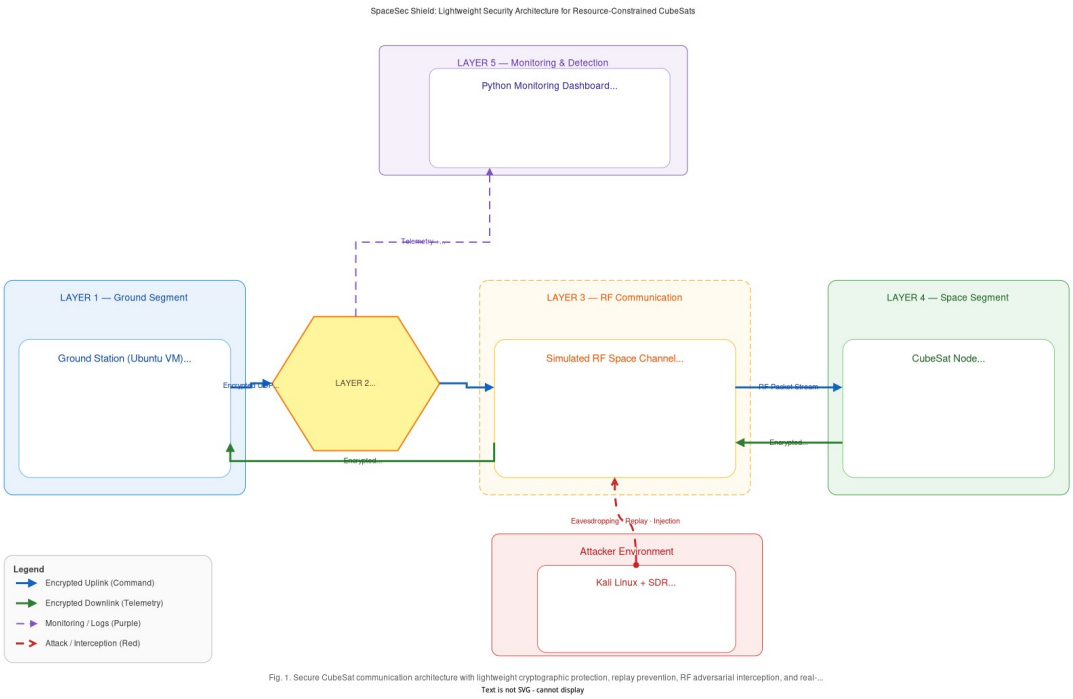

Figure 1. SpaceSec Shield security architecture for Ground Station and Satellite Node communication.

6.1 Secure Session Establishment

The handshake uses ephemeral X25519 key pairs. Each peer generates a fresh key pair and exchanges only the raw 32-byte public key. Both peers then derive the same ECDH shared secret and pass it through HKDF-SHA256 using project-specific salt and info labels. The output is a 32-byte session key used by ChaCha20-Poly1305.

6.2 Packet Format

The packet header is transmitted in clear form but authenticated as AAD. This allows the receiver to inspect routing and freshness fields while preventing attackers from modifying them undetected. The payload is encrypted, and the Poly1305 tag authenticates both the header and ciphertext.

Field	Size	Security Role
Session ID	4 bytes	Identifies the secure session and is authenticated as AAD.
Timestamp	8 bytes	Supports freshness validation and is authenticated as AAD.

Nonce	12 bytes	Unique AEAD nonce per packet; included in AAD and used as the ChaCha20-Poly1305 IV.
Ciphertext	Variable	Encrypted command payload.
Poly1305 Tag	16 bytes	Authentication tag covering AAD and ciphertext.

6.3 Security Rules

- Every session key and nonce pair must be used only once.
- The receiver checks timestamp freshness before accepting commands.
- The receiver rejects duplicate nonces for the same session.
- The receiver verifies the AEAD tag before using plaintext.
- Tampered, forged, replayed, and expired packets are discarded before command execution.

7. Implementation and Simulation Environment

The prototype was implemented in C++ on Kali Linux running inside Oracle VirtualBox. OpenSSL was used for X25519, HKDF-SHA256, and ChaCha20-Poly1305 through the EVP API. UDP sockets were used to simulate the command channel between a Ground Station and Satellite Node on localhost. Wireshark was used for packet capture and payload inspection.

Component	Value
Operating System	Kali Linux 2025.3 in Oracle VirtualBox
Programming Language	C++17
Compiler	g++ with -std=c++17
Cryptographic Library	OpenSSL 3.6.2
Network Protocol	UDP over 127.0.0.1
Listening Port	9000
Packet Analysis Tool	Wireshark 4.4.4
Build Flags	-lssl -lcrypto

The following compile commands were used for the final UDP demonstration:

```
g++ -std=c++17 secure_satellite_node.cpp handshake.cpp crypto.cpp packet.cpp
anti_replay.cpp -o satellite -lssl -lcrypto
```

```
g++ -std=c++17 secure_ground_station.cpp handshake.cpp crypto.cpp packet.cpp -o ground
-lssl -lcrypto
```

8. Attack Scenarios

Four attack categories were addressed in the implementation and testing.

Attack Scenario	Threat Description	Defense Implemented
Eavesdropping	An attacker captures packets and tries to read satellite commands.	Command payload is encrypted using ChaCha20-Poly1305. Wireshark did not reveal plaintext commands.
Replay Attack	An attacker resends a previously valid encrypted command.	Timestamp freshness and nonce cache reject duplicate nonces and expired packets.
Command Injection / Tampering	An attacker modifies ciphertext, tag, or header fields.	AEAD verification fails, so the packet is discarded before command execution.
Spoofing	An attacker attempts to send packets without the valid session key.	Packets without a valid Poly1305 tag under the session key fail authentication.

9. Testing and Results

Testing was performed in layers. The following table summarizes the main test cases and observed results.

Test Case	Expected Result	Actual Result	Status
Crypto verification	Valid packet decrypts; tampered ciphertext, tag, and AAD are rejected.	All tampering cases were rejected.	PASS
Packet structure	Serialized packet decrypts; modified header, payload, or tag fails.	Header, payload, and tag tampering were rejected.	PASS
Anti-replay	Fresh packet accepted; duplicate nonce and expired timestamp rejected.	Replay and expired timestamp were rejected.	PASS
Handshake	Ground and Satellite derive matching 32-byte session keys.	Both sides derived identical session keys.	PASS

End-to-end flow	Secure command accepted; replay, tampered, and expired packets rejected.	All expected accept/reject decisions occurred.	PASS
UDP demo	Ground sends UDP secure packets; Satellite validates them.	Valid command accepted and attack packets rejected.	PASS
Wireshark evidence	Plaintext command should not appear in captured packet bytes.	Search for CMD:CAPTURE_IMAGE returned no packet matches.	PASS

9.1 Cryptographic Verification

The first test verified the AEAD behavior of ChaCha20-Poly1305. A valid command decrypted successfully, while changes to the ciphertext, authentication tag, or AAD caused verification failure.

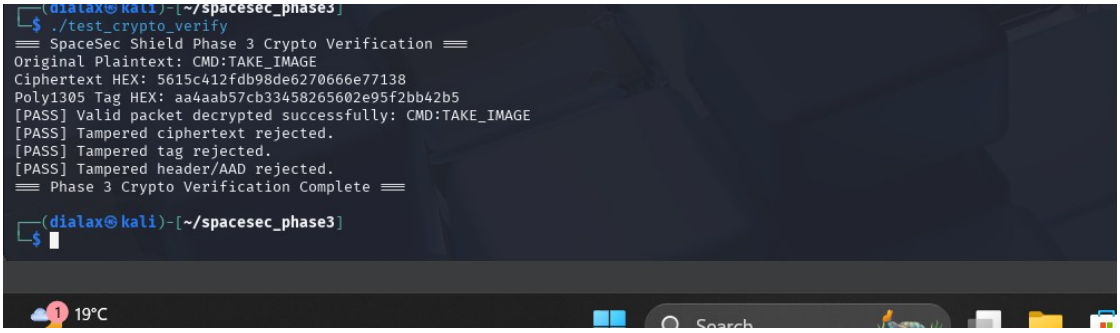

Figure 2. Crypto verification: valid decryption and rejection of tampered ciphertext, tag, and AAD.

9.2 Packet Structure Validation

The packet test validated that the serialized format correctly preserved the header, ciphertext, and tag. It also confirmed that modifications to any protected packet component were detected.

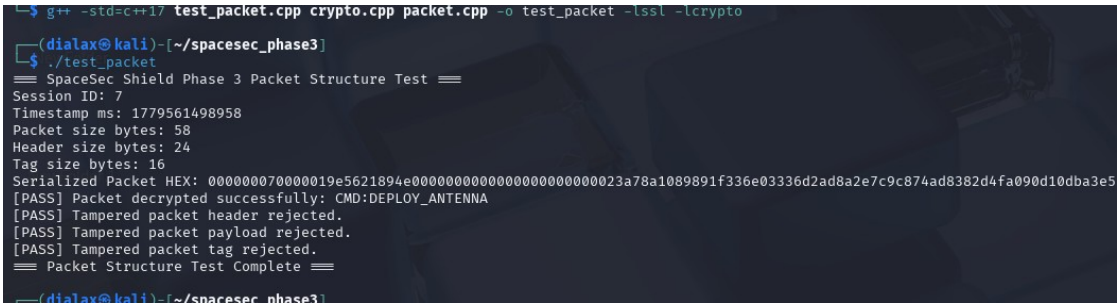

Figure 3. Packet structure validation: protected packet decrypts successfully and tampered components are rejected.

9.3 Anti-Replay Validation

The anti-replay test confirmed that the first fresh packet was accepted, the same packet resent with the same nonce was rejected, and a packet with an expired timestamp was rejected.

Figure 4. Anti-replay validation: duplicate nonce, expired timestamp, and tampered packet rejection.

9.4 Secure Handshake Validation

The handshake test demonstrated that the Ground Station and Satellite Node generated ephemeral X25519 public keys, derived the same 32-byte ECDH shared secret, and produced identical 32-byte HKDF session keys.

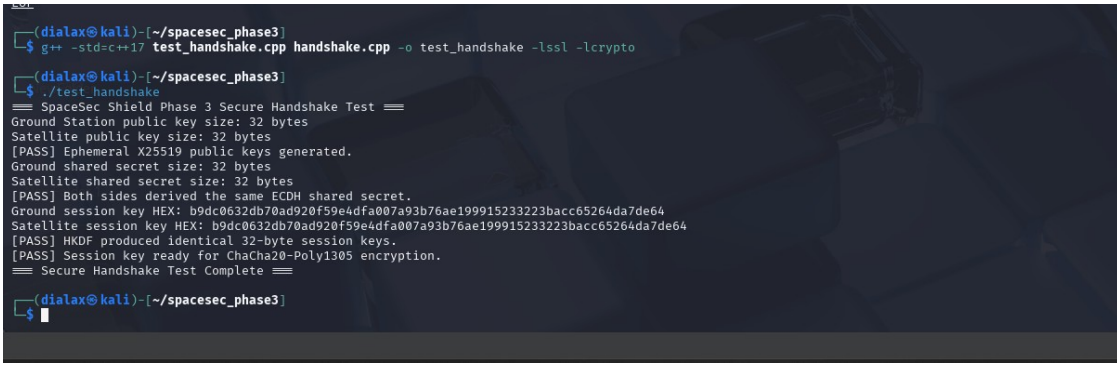

Figure 5. Secure handshake: X25519 shared secret and HKDF session key agreement.

9.5 End-to-End Secure Flow

The integrated end-to-end test replaced the fixed test key with the real session key produced by X25519 and HKDF. The Satellite Node accepted only the authenticated command and rejected replayed, tampered, and expired packets.

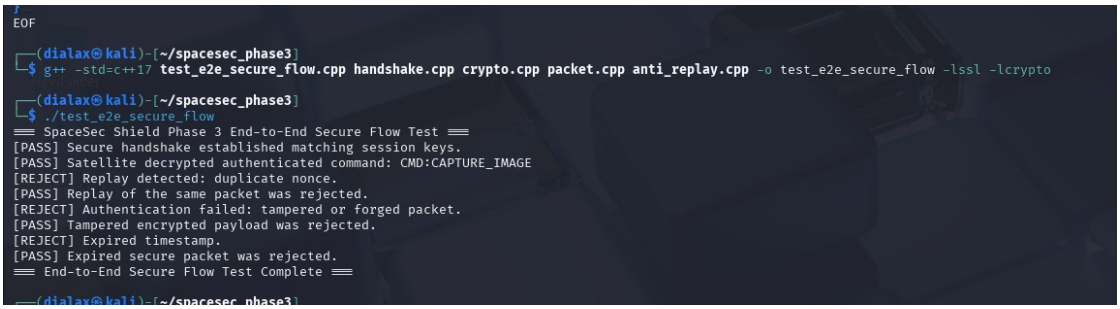

Figure 6. End-to-end secure flow using the derived session key.

9.6 Secure UDP Demonstration

The final runtime demo used two terminals. The Satellite Node listened on UDP port 9000 and the Ground Station sent four packet types: a valid encrypted command, a replayed packet, a tampered encrypted payload, and an expired timestamp packet. The Satellite Node accepted only the valid command.

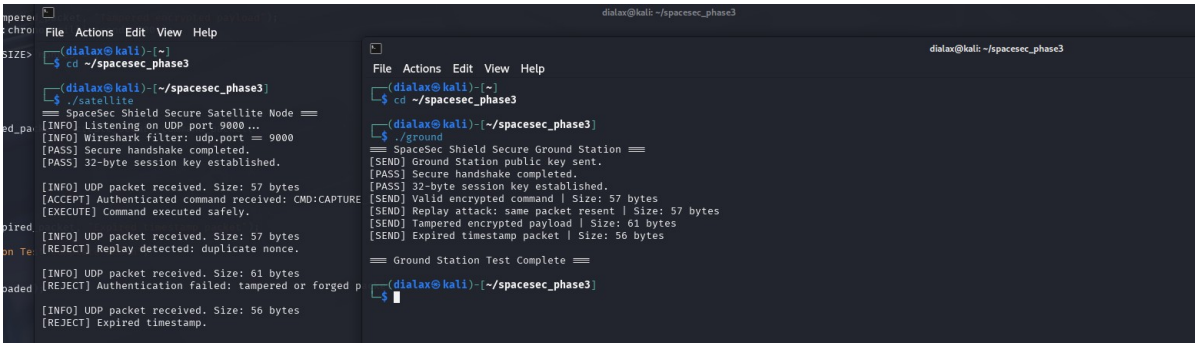

Figure 7. Live UDP secure demo showing Ground Station transmissions and Satellite Node accept/reject decisions.

9.7 Wireshark Packet Analysis

Wireshark was used to capture UDP traffic on the loopback interface using the display filter udp.port == 9000. Packets 1 and 2 represent 32-byte public-key exchange messages. Packet 3 represents the valid encrypted command packet with a 57-byte data field. The packet bytes show non-readable data rather than the plaintext command string.

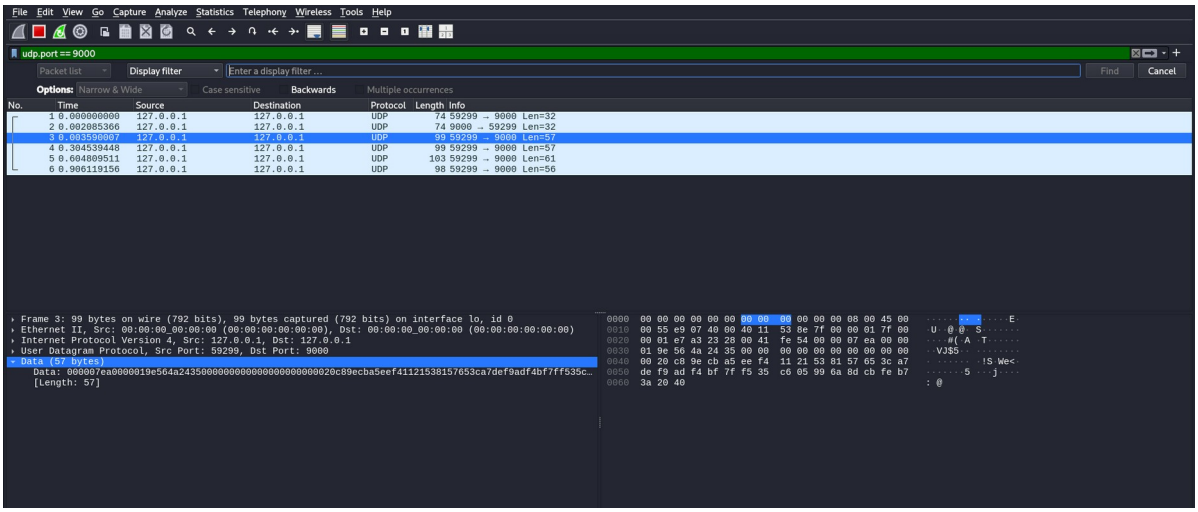

Figure 8. Wireshark capture of UDP packet No. 3 showing encrypted Data bytes with no plaintext command visible.

A packet-bytes string search for CMD:CAPTURE_IMAGE returned no matches across the captured packets. This confirms that the command was not transmitted as plaintext in the UDP payload.

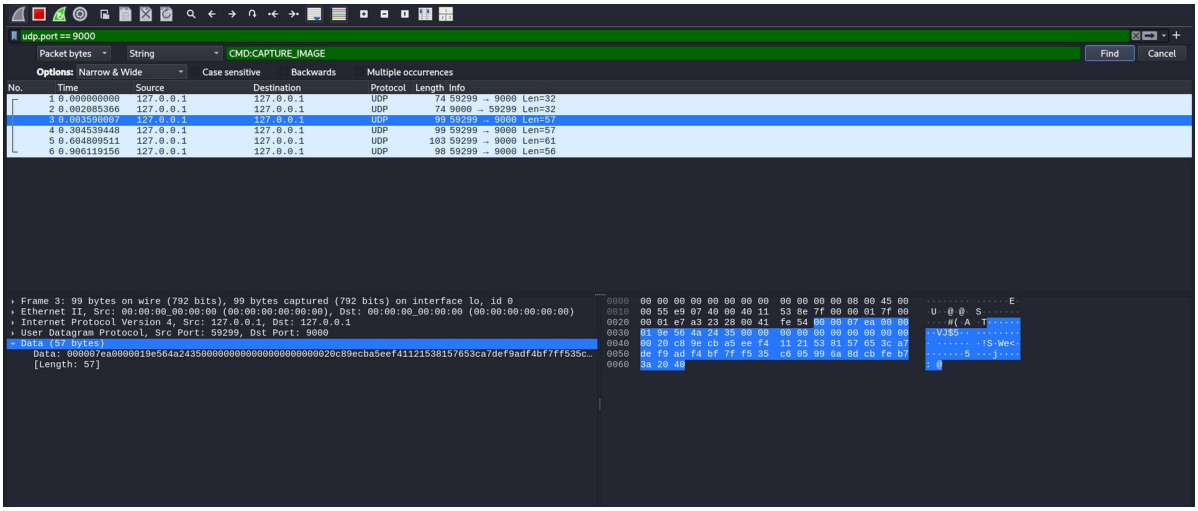

Figure 9. Wireshark packet-bytes search: no packet contained the plaintext command string.

10. Challenges

The following challenges were encountered and addressed during implementation.

Challenge	Impact	Resolution
Nonce uniqueness	Nonce reuse under the same session key would break ChaCha20-Poly1305 security.	A structured 12-byte nonce was generated per packet and tracked by the receiver.

Header protection	Clear headers could be modified if not authenticated.	Header bytes were passed as AAD so changes cause tag verification failure.
Replay rejection	A captured valid packet could be resent successfully without duplicate detection.	A seen-nonce set per session was implemented with a timestamp freshness window.
Debugging encrypted traffic	Encrypted payloads are intentionally unreadable in Wireshark.	Terminal logs were paired with Wireshark evidence to correlate packet order and security decisions.
Resource-aware algorithm selection	Heavy cryptographic stacks may be unsuitable for CubeSat-class processors.	ChaCha20-Poly1305 was selected as a lightweight AEAD construction.

11. Conclusion

Phase 3 successfully implemented the SpaceSec Shield security layer for CubeSat-style communication. The final prototype establishes a secure session using ephemeral X25519 ECDH, derives a 32-byte session key with HKDF-SHA256, encrypts and authenticates command payloads with ChaCha20-Poly1305, and rejects replayed or tampered packets using timestamp and nonce verification.

The evidence demonstrates that security goals were achieved. Valid encrypted commands were accepted and executed safely. Replayed packets were rejected due to duplicate nonces. Tampered packets were rejected by AEAD authentication failure. Expired packets were rejected through timestamp freshness checks. Wireshark analysis confirmed that plaintext commands were not visible in captured UDP packet bytes.

The implementation achieves a practical balance between security, performance, and resource awareness. It provides confidentiality, integrity, authenticity, and replay resistance without adding external hardware or heavyweight protocols.

12. Future Work

- Integrate Phase 4 penetration testing to attack the secured channel using replay, injection, spoofing, and tampering scenarios.
- Add automated packet counters and nonce generation based on monotonic counters to avoid manual nonce assignment.

- Add session rekeying after a fixed number of packets or time interval.
- Implement persistent logging for accepted and rejected packets to support mission audit trails.
- Extend the prototype to support multi-satellite constellations with per-node session identifiers.
- Compare measured CPU time and packet latency for ChaCha20-Poly1305 versus AES-128-GCM on embedded hardware.
- Add a lightweight dashboard in Phase 5 to visualize secure packets, attack attempts, and rejection reasons.

13. References

Cal Poly CubeSat Program. (2022). CubeSat Design Specification Rev. 14.1. https://www.cubesat.org/

Dworkin, M. (2007). Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC (NIST SP 800-38D). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-38D

Krawczyk, H., & Eronen, P. (2010). HMAC-based Extract-and-Expand Key Derivation Function (HKDF) (RFC 5869). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc5869

Langley, A., Hamburg, M., & Turner, S. (2016). Elliptic curves for security (RFC 7748). Internet Engineering Task Force. https://www.rfc-editor.org/rfc/rfc7748

NASA. (2017). CubeSat 101: Basic concepts and processes for first-time CubeSat developers. National Aeronautics and Space Administration. https://www.nasa.gov/

Nir, Y., & Langley, A. (2018). ChaCha20 and Poly1305 for IETF protocols (RFC 8439). Internet Engineering Task Force. https://datatracker.ietf.org/doc/html/rfc8439

OpenSSL Project. (n.d.). EVP ChaCha20 and ChaCha20-Poly1305 documentation. https://docs.openssl.org/

SpaceSec Shield Project Team. (2026). Phase 3 - Security Layer: Implementation Task. Internal project document.

NTP 2026. (2026). SpaceSec Shield final report guidelines. Internal competition guideline document.